

Title:	A Unified Executors Proposal for C++
Authors:	Jared Hoberock, jhoberock@nvidia.com Michael Garland, mgarland@nvidia.com Chris Kohlhoff, chris@kohlhoff.com Chris Mysen, mysen@google.com Carter Edwards, hcedwar@sandia.gov Gordon Brown, gordon@codeplay.com
Other Contributors:	Hans Boehm, hboehm@google.com Thomas Heller, thom.heller@gmail.com Lee Howes, lwh@fb.com Bryce Lelbach, brycelelbach@gmail.com Hartmut Kaiser, hartmut.kaiser@gmail.com Bryce Lelbach, brycelelbach@gmail.com Gor Nishanov, gorn@microsoft.com Thomas Rodgers, rodgert@twroddgers.com David Hollman, dshollm@sandia.gov Michael Wong, michael@codeplay.com
Document Number:	P0443R9
Date:	2018-10-08
Audience:	SG1 - Concurrency and Parallelism, LEWG
Reply-to:	sg1-exec@googlegroups.com
Abstract:	This paper proposes a programming model for executors, which are modular components for creating execution. The design of this proposal is described in paper P0761 .

0.1 Changelog

0.1.1 Revision 9

As directed by the SG1/LEWG straw poll taken during the 2018 Bellevue executors meeting, we have separated The Unified Executors programming model proposal into two papers. This paper contains material related to one-way execution which the authors hope to standardize with C++20 as suggested by the Bellevue poll. [P1244](#) contains remaining material related to dependent execution. We expect P1244 to evolve as committee consensus builds around a design for dependent execution.

This revision also contains bug fixes to the `allocator_t` property which were originally scheduled for Revision 7 but were inadvertently omitted.

0.1.2 Revision 8

Revision 8 of this proposal makes interface-changing properties such as `oneway` mutually exclusive in order to simplify implementation requirements for executor adaptors such as polymorphic executors. Additionally, this revision clarifies wording regarding execution agent lifetime.

0.1.3 Revision 7

Revision 7 of this proposal corrects wording bugs discovered by the authors after Revision 6's publication.

- Enhanced `static_query_v` to result in a default property value for executors which do not provide a `query` function for the property of interest
- Revise `then_execute` and `bulk_then_execute`'s operational semantics to allow user functions to handle incoming exceptions thrown by preceding execution agents
- Introduce `exception_arg` to disambiguate the user function's exceptional overload from its nonexceptional overload in `then_execute` and `bulk_then_execute`

0.1.4 Revision 6

Revision 6 of this proposal corrects bugs and omissions discovered by the authors after Revision 5's publication, and introduces an enhancement improving the safety of the design.

- Enforce mutual exclusion of behavioral properties via the type system instead of via convention
- Introduce missing `execution::require` adaptations
- Allow executors to opt-out of invoking factory functions when appropriate
- Various bug fixes and corrections

0.1.5 Revision 5

Revision 5 of this proposal responds to feedback requested during the 2017 Albuquerque ISO C++ Standards Committee meeting and introduces changes which allow properties to better interoperate with polymorphic executor wrappers and also simplify `execution::require`'s behavior.

- Defined general property type requirements
- Elaborated specification of standard property types
- Simplified `execution::require`'s specification
- Enhanced polymorphic executor wrapper

- Templated execution: `::executor<SupportableProperties...>`
- Introduced `prefer_only` property adaptor
- Responded to Albuquerque feedback
 - From SG1
 - Execution contexts are now optional properties of executors
 - Eliminated ill-specified caller-agent forward progress properties
 - Elaborated `Future`'s requirements to incorporate forward progress
 - Reworded operational semantics of execution functions to use similar language as the blocking properties
 - Elaborated `static_thread_pool`'s specification to guarantee that threads in the `bool` boost-block their work
 - Elaborated operational semantics of execution functions to note that forward progress guarantees are specific to the concrete executor type
 - From LEWG
 - Eliminated named `BaseExecutor` concept
 - Simplified general executor requirements
 - Enhanced the `OneWayExecutor` introductory paragraph
 - Eliminated `has_*_member` type traits
- Minor changes
 - Renamed TS namespace from `concurrency_v2` to `executors_v1`
 - Introduced `static_query_v` enabling static queries
 - Eliminated unused `property_value` trait
 - Eliminated the names `allocator_wrapper_t` and `default_allocator`

0.1.6 Revision 4

- Specified the guarantees implied by `bulk_sequenced_execution`, `bulk_parallel_execution`, and `bulk_unsequenced_execution`

0.1.7 Revision 3

- Introduced `execution::query()` for executor property introspection
- Simplified the design of `execution::prefer()`
- `oneway`, `twoway`, `single`, and `bulk` are now `require()-only` properties
- Introduced properties allowing executors to opt into adaptations that add blocking semantics
- Introduced properties describing the forward progress relationship between caller and agents
- Various minor improvements to existing functionality based on prototyping

0.1.8 Revision 2

- Separated wording from explanatory prose, now contained in paper [P0761](#)
- Applied the simplification proposed by paper [P0688](#)

0.1.9 Revision 1

- Executor category simplification
- Specified executor customization points in detail
- Introduced new fine-grained executor type traits
 - Detectors for execution functions
 - Traits for introspecting cross-cutting concerns
 - Introspection of mapping of agents to threads
 - Introspection of execution function blocking behavior
- Allocator support for single agent execution functions
- Renamed `thread_pool` to `static_thread_pool`
- New introduction

0.1.10 Revision 0

- Initial design

1 Proposed Wording

1.1 Execution Support Library

1.1.1 General

This Clause describes components supporting execution of function objects [function.objects].

(The following definition appears in working draft N4762 [thread.req.lockable.general])

An *execution agent* is an entity such as a thread that may perform work in parallel with other execution agents. [Note: Implementations or users may introduce other kinds of agents such as processes or thread-pool tasks. *--end note*] The calling agent is determined by context; e.g., the calling thread that contains the call, and so on.

An execution agent invokes a function object within an *execution context* such as the calling thread or thread-pool. An *executor* submits a function object to an execution context to be invoked by an execution agent within that execution context. [Note: Invocation of the function object may be

inlined such as when the execution context is the calling thread, or may be scheduled such as when the execution context is a thread-pool with task scheduler. *--end note*] An executor may submit a function object with *execution properties* that specify how the submission and invocation of the function object interacts with the submitting thread and execution context, including forward progress guarantees [intro.progress].

For the intent of this library and extensions to this library, the *lifetime of an execution agent* begins before the function object is invoked and ends after this invocation completes, either normally or having thrown an exception.

1.1.2 Header <execution> synopsis

```
namespace std {
namespace execution {

    // Customization points:

namespace {
    constexpr unspecified require = unspecified;
    constexpr unspecified prefer = unspecified;
    constexpr unspecified query = unspecified;
}

    // Customization point type traits:

template<class Executor, class... Properties> struct can_require;
template<class Executor, class... Properties> struct can_prefer;
template<class Executor, class Property> struct can_query;

template<class Executor, class... Properties>
    constexpr bool can_require_v = can_require<Executor, Properties...>::value;
template<class Executor, class... Properties>
    constexpr bool can_prefer_v = can_prefer<Executor, Properties...>::value;
template<class Executor, class Property>
    constexpr bool can_query_v = can_query<Executor, Property>::value;

    // Associated execution context property:

struct context_t;

constexpr context_t context;

    // Interface-changing properties:

struct oneway_t;
struct bulk_oneway_t;

constexpr oneway_t oneway;
constexpr bulk_oneway_t bulk_oneway;

    // Blocking properties:

struct blocking_t;

constexpr blocking_t blocking;

    // Properties to allow adaptation of blocking and directionality:

struct blocking_adaptation_t;

constexpr blocking_adaptation_t blocking_adaptation;

    // Properties to indicate if submitted tasks represent continuations:

struct relationship_t;

constexpr relationship_t relationship;

    // Properties to indicate likely task submission in the future:

struct outstanding_work_t;

constexpr outstanding_work_t outstanding_work;

    // Properties for bulk execution guarantees:

struct bulk_guarantee_t;

constexpr bulk_guarantee_t bulk_guarantee;

    // Properties for mapping of execution on to threads:

struct mapping_t;

constexpr mapping_t mapping;

    // Memory allocation properties:

template <typename ProtoAllocator>
struct allocator_t;

constexpr allocator_t<void> allocator;

    // Executor type traits:
```

```

template<class Executor> struct is_oneway_executor;
template<class Executor> struct is_bulk_oneway_executor;

template<class Executor> constexpr bool is_oneway_executor_v = is_oneway_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_oneway_executor_v = is_bulk_oneway_executor<Executor>::value;

template<class Executor> struct executor_shape;
template<class Executor> struct executor_index;

template<class Executor> using executor_shape_t = typename executor_shape<Executor>::type;
template<class Executor> using executor_index_t = typename executor_index<Executor>::type;

// Polymorphic executor support:

class bad_executor;

template <class InterfaceProperty, class... SupportableProperties>
    using executor = typename InterfaceProperty::template
        polymorphic_executor_type<InterfaceProperty, SupportableProperties...>;
template<class Property> struct prefer_only;

} // namespace execution
} // namespace std

```

1.2 Requirements

1.2.1 Customization point objects

(The following text has been adapted from the draft *Ranges Technical Specification*.)

A *customization point object* is a function object (C++ Std, [function.objects]) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.

The type of a customization point object shall satisfy the requirements of `CopyConstructible` (C++Std [copyconstructible]) and `Destructible` (C++Std [destructible]).

All instances of a specific customization point object type shall be equal.

Let t be a (possibly `const`) customization point object of type τ , and $\text{args}...$ be a parameter pack expansion of some parameter pack $\text{Args}...$. The customization point object t shall be invocable as $t(\text{args}...)$ when the types of $\text{Args}...$ meet the requirements specified in that customization point object's definition. Otherwise, τ shall not have a function call operator that participates in overload resolution.

Each customization point object type constrains its return type to satisfy some particular type requirements.

The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.

[*Note:* Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results in invoking a user-defined function found by argument dependent name lookup (C++Std [basic.lookup.argdep]). To preclude such an expression resulting in invoking an unconstrained functions with the same name in namespace `std`, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace `std`. When the deleted overloads are viable, user-defined overloads must be more specialized (C++Std [temp.func.order]) to be used by a customization point object. *--end note*]

1.2.2 ProtoAllocator requirements

A type A meets the `ProtoAllocator` requirements if A is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where U is an object type. [*Note:* For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. *--end note*] No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

1.2.3 General requirements on executors

An executor type shall satisfy the requirements of `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `EqualityComparable` (C++Std [equalitycomparable]).

None of these concepts' operations, nor an executor type's swap operations, shall exit via an exception.

None of these concepts' operations, nor an executor type's associated execution functions, associated query functions, or other member functions defined in executor type requirements, shall introduce data races as a result of concurrent invocations of those functions from different threads.

For any two (possibly `const`) values x_1 and x_2 of some executor type X , $x_1 == x_2$ shall return `true` only if $x_1.\text{query}(p) == x_2.\text{query}(p)$ for every property p where both $x_1.\text{query}(p)$ and $x_2.\text{query}(p)$ are well-formed and result in a non-void type that is `EqualityComparable` (C++Std [equalitycomparable]). [*Note:* The above requirements imply that $x_1 == x_2$ returns `true` if x_1 and x_2 can be interchanged with identical effects. An executor may conceptually contain additional properties which are not exposed by a named property type that can be observed via `execution::query`; in this case, it is up to the concrete executor implementation to decide if these properties affect equality. Returning `false` does not necessarily imply that the effects are not identical. *--end note*]

An executor type's destructor shall not block pending completion of the submitted function objects. [*Note:* The ability to wait for completion of submitted function objects may be provided by the associated execution context. *--end note*]

1.2.4 OneWayExecutor requirements

A type `X` satisfies the `OneWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the Table below.

[*Note:* `OneWayExecutorS` provides fire-and-forget semantics without a channel for awaiting the completion of a submitted function object and obtaining its result. *--end note*]

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`
- `f` denotes a function object of type `F&&` invocable as `cf()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.execute(f)</code>	<code>void</code>	Evaluates <code>DECAY_COPY(std::forward<F>(f))</code> on the calling thread to create <code>cf</code> that will be invoked at most once by an execution agent. May block pending completion of this invocation. Synchronizes with [intro.multithread] the invocation of <code>f</code> . Shall not propagate any exception thrown by the function object or any other function submitted to the executor. [<i>Note:</i> The treatment of exceptions thrown by one-way submitted functions and the forward progress guarantee of the associated execution agent(s) are implementation defined. <i>--end note.</i>]

1.2.5 BulkOneWayExecutor requirements

The `BulkOneWayExecutor` requirements specify requirements for executors which submit a function object to be invoked multiple times without a channel for awaiting the completion of the submitted function object invocations and obtaining their result. [*Note:* That is, the executor provides fire-and-forget semantics. *--end note*]

A type `X` satisfies the `BulkOneWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the Table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly `const`) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`,
- `f` denotes a function object of type `F&&` invocable as `cf(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements,

Expression	Return Type	Operational semantics
<code>x.bulk_execute(f, n, sf)</code>	<code>void</code>	Evaluates <code>DECAY_COPY(std::forward<F>(f))</code> on the calling thread to create a function object <code>cf</code> . [<i>Note:</i> * Additional copies of <code>cf</code> may subsequently be created. * <i>--end note</i>] For each value of <code>i</code> in shape <code>n</code> <code>cf(i, s)</code> (or copy of <code>cf</code>) will be invoked at most once by an execution agent that is unique for each value of <code>i</code> . <code>sf()</code> will be invoked at most once to produce value <code>s</code> before any invocation of <code>cf</code> . May block pending completion of one or more invocations of <code>cf</code> . Synchronizes with (C++Std [intro.multithread]) the invocations of <code>f</code> . Shall not propagate any exception thrown by <code>cf</code> or any other function submitted to the executor. [<i>Note:</i> The treatment of exceptions thrown by bulk one-way submitted functions and the forward progress guarantee of the associated execution agent(s) are implementation defined. <i>--end note.</i>]

1.3 Executor customization points

Executor customization points are functions which adapt an executor's properties. Executor customization points enable uniform use of executors in generic contexts.

When an executor customization point named *NAME* invokes a free execution function of the same name, overload resolution is performed in a context that includes the declaration `void NAME(auto&... args) = delete;`, where `sizeof...(args)` is the arity of the free execution function. This context also does not include a declaration of the executor customization point.

[*Note:* This provision allows executor customization points to invoke the executor's free, non-member execution function of the same name without recursion. *--end note*]

Whenever `std::execution::NAME(ARGS)` is a valid expression, that expression satisfies the syntactic requirements for the free execution function named *NAME* with arity `sizeof...(ARGS)` with that free execution function's semantics.

1.3.1 require

```
namespace {
    constexpr unspecified require = unspecified;
}
```

The name `require` denotes a customization point. The effect of the expression `std::execution::require(E, P0, Pn...)` for some expressions *E* and *P0*, and where *Pn...* represents *N* expressions (where *N* is 0 or more), is equivalent to:

- If *N* == 0, *P0::is_requirable* is true, and the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>> == decay_t<decltype(P0)>::value()` is a well-formed constant expression with value `true`, *E*.
- If *N* == 0, *P0::is_requirable* is true, and the expression `(E).require(P0)` is well-formed, `(E).require(P0)`.
- If *N* == 0, *P0::is_requirable* is true, and the expression `require(E, P0)` is well-formed, `require(E, P0)`.
- If *N* > 0 and the expression `std::execution::require( std::execution::require(E, P0), Pn...)` is well formed, `std::execution::require( std::execution::require(E, P0), Pn...)`.
- Otherwise, `std::execution::require(E, P0, Pn...)` is ill-formed.

1.3.2 prefer

```
namespace {
    constexpr unspecified prefer = unspecified;
}
```

The name `prefer` denotes a customization point. The effect of the expression `std::execution::prefer(E, P0, Pn...)` for some expressions *E* and *P0*, and where *Pn...* represents *N* expressions (where *N* is 0 or more), is equivalent to:

- If *N* == 0, *P0::is_preferable* is true, and the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>> == decay_t<decltype(P0)>::value()` is a well-formed constant expression with value `true`, *E*.
- If *N* == 0, *P0::is_preferable* is true, and the expression `(E).require(P0)` is well-formed, `(E).require(P0)`.
- If *N* == 0, *P0::is_preferable* is true, and the expression `prefer(E, P0)` is well-formed, `prefer(E, P0)`.
- If *N* == 0 and *P0::is_preferable* is true, *E*.
- If *N* > 0 and the expression `std::execution::prefer( std::execution::prefer(E, P0), Pn...)` is well formed, `std::execution::prefer( std::execution::prefer(E, P0), Pn...)`.
- Otherwise, `std::execution::require(E, P0, Pn...)` is ill-formed.

1.3.3 query

```
namespace {
    constexpr unspecified query = unspecified;
}
```

The name `query` denotes a customization point. The effect of the expression `std::execution::query(E, P)` for some expressions *E* and *P* is equivalent to:

- If the expression `decay_t<decltype(P)>::template static_query_v<decay_t<decltype(E)>> is a well-formed constant expression, decay_t<decltype(P)>::template static_query_v<decay_t<decltype(E)>>.`
- If the expression `(E).query(P)` is well-formed, `(E).query(P)`.
- If the expression `query(E, P)` is well-formed, `query(E, P)`.
- Otherwise, `std::execution::query(E, P)` is ill-formed.

1.3.4 Customization point type traits

```
template<class Executor, class... Properties> struct can_require;
template<class Executor, class... Properties> struct can_prefer;
template<class Executor, class Property> struct can_query;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
	The expression <code>std::execution::require(</code>	

<pre>template<class T> struct can_require</pre>	<pre>declval<const Executor>(), declval<Properties>()...) is well formed.</pre>	τ is a complete type.
<pre>template<class T> struct can_prefer</pre>	The expression <code>std::execution::prefer(declval<const Executor>(), declval<Properties>()...)</code> is well formed.	τ is a complete type.
<pre>template<class T> struct can_query</pre>	The expression <code>std::execution::query(declval<const Executor>(), declval<Property>())</code> is well formed.	τ is a complete type.

1.4 Executor properties

1.4.1 In general

An executor's behavior in generic contexts is determined by a set of executor properties, and each executor property imposes certain requirements on the executor.

Given an existing executor, a related executor with different properties may be created by invoking the `require` member or non-member functions. These functions behave according the Table below. In the Table below, x denotes a (possibly `const`) executor object of type x , and p denotes a (possibly `const`) property object.

[*Note:* As a general design note properties which define a mutually exclusive pair, that describe an enabled or non-enabled behaviour follow the convention of having the same property name for both with the `not_` prefix to the property for the non-enabled behaviour. *--end note*]

Expression	Comments
<pre>x.require(p) require(x,p)</pre>	Returns an executor object with the requested property p added to the set. All other properties of the returned executor are identical to those of x, except where those properties are described below as being mutually exclusive to p. In this case, the mutually exclusive properties are implicitly removed from the set associated with the returned executor. The expression is ill formed if an executor is unable to add the requested property.

The current value of an executor's properties can be queried by invoking the `query` function. This function behaves according the Table below. In the Table below, x denotes a (possibly `const`) executor object of type x , and p denotes a (possibly `const`) property object.

Expression	Comments
<pre>x.query(p)</pre>	Returns the current value of the requested property p. The expression is ill formed if an executor is unable to return the requested property.

1.4.2 Requirements on properties

A property type P shall provide:

- A nested constant expression named `is_requirable` of type `bool`, usable as `P::is_requirable`.
- A nested constant expression named `is_preferable` of type `bool`, usable as `P::is_preferable`.

[*Note:* These constants are used to determine whether the property can be used with the `require` and `prefer` customization points, respectively. *--end note*]

A property type P may provide a nested type `polymorphic_query_result_type` that satisfies the `CopyConstructible` and `Destructible` requirements. If `P::is_requirable == true` OR `P::is_preferable == true`, `polymorphic_query_result_type` shall also satisfy the `DefaultConstructible` requirements. [*Note:* When present, this type allows the property to be used with a polymorphic executor wrapper. *--end note*]

A property type P may provide:

- A nested variable template `static_query_v`, usable as `P::static_query_v<Executor>`. This may be conditionally present.
- A member function `value()`.

If both `static_query_v` and `value()` are present, they shall return the same type and this type shall satisfy the `EqualityComparable` requirements.

[*Note*: These are used to determine whether invoking `require` would result in an identity transformation. *--end note*]

1.4.3 Query-only properties

1.4.3.1 Associated execution context property

```
struct context_t
{
    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = any; // TODO: alternatively consider void*, or simply omitting the type.

    template<class Executor>
        static constexpr decltype(auto) static_query_v
            = Executor::query(context_t());
};
```

The `context_t` property can be used only with `query`, which returns the execution context associated with the executor.

The value returned from `execution::query(e, context_t)`, where `e` is an executor, shall not change between invocations.

1.4.4 Interface-changing properties

```
struct oneway_t;
struct bulk_oneway_t;
```

The interface-changing properties conform to the following specification:

```
struct S
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = false;

    template<class... SupportableProperties>
        class polymorphic_executor_type;

    using polymorphic_query_result_type = bool;

    template<class Executor>
        static constexpr bool static_query_v
            = see-below;

    static constexpr bool value() const { return true; }
};
```

Property	Requirements
<code>oneway_t</code>	The executor type satisfies the <code>OneWayExecutor</code> requirements.
<code>bulk_oneway_t</code>	The executor type satisfies the <code>BulkOneWayExecutor</code> requirements.

`S::static_query_v<Executor>` is true if and only if `Executor` fulfills `S`'s requirements.

The `oneway_t` and `bulk_oneway_t` properties are mutually exclusive.

1.4.4.1 Polymorphic wrappers

In several places in this section the operation `CONTAINS_PROPERTY(p, pn)` is used. All such uses mean `std::disjunction_v<std::is_same<p, pn>...>`.

In several places in this section the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is used. All such uses mean the first type `P` in the parameter pack `pn` for which `std::is_convertible_v<p, P>` is true. If no such type `P` exists, the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is ill-formed.

The nested class template `S::polymorphic_executor_type` conforms to the following specification.

```
template <class... SupportableProperties>
class polymorphic_executor_type
{
public:
    // construct / copy / destroy:

    polymorphic_executor_type() noexcept;
    polymorphic_executor_type(nullptr_t) noexcept;
    polymorphic_executor_type(const polymorphic_executor_type& e) noexcept;
    polymorphic_executor_type(polymorphic_executor_type&& e) noexcept;
    template<class... OtherSupportableProperties>
        polymorphic_executor_type(polymorphic_executor_type<OtherSupportableProperties...> e);
    template<class... OtherSupportableProperties>
```



```

    polymorphic_executor_type(polymorphic_executor_type<OtherSupportableProperties...> e) = delete;

    polymorphic_executor_type& operator=(const polymorphic_executor_type& e) noexcept;
    polymorphic_executor_type& operator=(polymorphic_executor_type&& e) noexcept;
    polymorphic_executor_type& operator=(nullptr_t) noexcept;

    ~polymorphic_executor_type();

    // polymorphic_executor_type modifiers:

    void swap(polymorphic_executor_type& other) noexcept;

    // polymorphic_executor_type operations:

    template <class Property>
    polymorphic_executor_type require(Property) const;

    template <class Property>
    typename Property::polymorphic_query_result_type query(Property) const;

    // polymorphic_executor_type capacity:

    explicit operator bool() const noexcept;

    // polymorphic_executor_type target access:

    const type_info& target_type() const noexcept;
    template<class Executor> Executor* target() noexcept;
    template<class Executor> const Executor* target() const noexcept;

    // polymorphic_executor_type casts:

    template<class... OtherSupportableProperties>
        polymorphic_executor_type<OtherSupportableProperties...> static_executor_cast() const;
};

// polymorphic_executor_type comparisons:

template <class... SupportableProperties>
    bool operator==(const polymorphic_executor_type<SupportableProperties...>& a, const polymorphic_executor_type<SupportableProperties...>& b) noexcept;
template <class... SupportableProperties>
    bool operator==(const polymorphic_executor_type<SupportableProperties...>& e, nullptr_t) noexcept;
template <class... SupportableProperties>
    bool operator==(nullptr_t, const polymorphic_executor_type<SupportableProperties...>& e) noexcept;
template <class... SupportableProperties>
    bool operator!=(const polymorphic_executor_type<SupportableProperties...>& a, const polymorphic_executor_type<SupportableProperties...>& b) noexcept;
template <class... SupportableProperties>
    bool operator!=(const polymorphic_executor_type<SupportableProperties...>& e, nullptr_t) noexcept;
template <class... SupportableProperties>
    bool operator!=(nullptr_t, const polymorphic_executor_type<SupportableProperties...>& e) noexcept;

// polymorphic_executor_type specialized algorithms:

template <class... SupportableProperties>
    void swap(polymorphic_executor_type<SupportableProperties...>& a, polymorphic_executor_type<SupportableProperties...>& b) noexcept;

template <class Property, class... SupportableProperties>
    polymorphic_executor_type prefer(const polymorphic_executor_type<SupportableProperties>& e, Property p);

```

The `polymorphic_executor_type` class satisfies the general requirements on executors.

[*Note:* To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more `polymorphic_executor_type` objects. *—end note*]

Each property type in the `SupportableProperties...` pack shall provide a nested type `polymorphic_query_result_type`.

The *target* is the executor object that is held by the wrapper.

1.4.4.1.1 `polymorphic_executor_type` constructors

```
polymorphic_executor_type() noexcept;
```

Postconditions: `!*this`.

```
polymorphic_executor_type(nullptr_t) noexcept;
```

Postconditions: `!*this`.

```
polymorphic_executor_type(const polymorphic_executor_type& e) noexcept;
```

Postconditions: `!*this` if `!e`; otherwise, `*this` targets `e.target()` or a copy of `e.target()`.

```
polymorphic_executor_type(polymorphic_executor_type&& e) noexcept;
```

Effects: If `!e`, `*this` has no target; otherwise, moves `e.target()` or move-constructs the target of `e` into the target of `*this`, leaving `e` in a valid state with an unspecified value.

```
template<class... OtherSupportableProperties>
    polymorphic_executor_type(polymorphic_executor_type<OtherSupportableProperties...> e);
```

Remarks: This function shall not participate in overload resolution unless: `* CONTAINS_PROPERTY(p, OtherSupportableProperties)` , where `p` is each property in `SupportableProperties...`

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

```
template<class... OtherSupportableProperties>
    polymorphic_executor_type(polymorphic_executor_type<OtherSupportableProperties...> e) = delete;
```

Remarks: This function shall not participate in overload resolution unless `CONTAINS_PROPERTY(p, OtherSupportableProperties)` is false for some property `p` in `SupportableProperties...`

1.4.4.1.2 polymorphic_executor_type assignment

```
polymorphic_executor_type& operator=(const polymorphic_executor_type& e) noexcept;
```

Effects: `polymorphic_executor_type(e).swap(*this)`.

Returns: `*this`.

```
polymorphic_executor_type& operator=(polymorphic_executor_type&& e) noexcept;
```

Effects: Replaces the target of `*this` with the target of `e`, leaving `e` in a valid state with an unspecified value.

Returns: `*this`.

```
polymorphic_executor_type& operator=(nullptr_t) noexcept;
```

Effects: `polymorphic_executor_type(nullptr).swap(*this)`.

Returns: `*this`.

1.4.4.1.3 polymorphic_executor_type destructor

```
~polymorphic_executor_type();
```

Effects: If `*this != nullptr`, releases shared ownership of, or destroys, the target of `*this`.

1.4.4.1.4 polymorphic_executor_type modifiers

```
void swap(polymorphic_executor_type& other) noexcept;
```

Effects: Interchanges the targets of `*this` and `other`.

1.4.4.1.5 polymorphic_executor_type operations

```
template <class Property>
    polymorphic_executor_type require(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_requirable` is well-formed and has the value `true`.

Returns: A polymorphic wrapper whose target is the result of `execution::require(e, p)`, where `e` is the target object of `*this`.

```
template <class Property>
    typename Property::polymorphic_query_result_type query(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)` is well-formed.

Returns: If `polymorphic_executor_type::query(e, p)` is well-formed, `static_cast<Property::polymorphic_query_result_type>(polymorphic_executor_type::query(e, p))`, where `e` is the target object of `*this`. Otherwise, `Property::polymorphic_query_result_type{}`.

1.4.4.1.6 polymorphic_executor_type capacity

```
explicit operator bool() const noexcept;
```

Returns: `true` if `*this` has a target, otherwise `false`.

1.4.4.1.7 polymorphic_executor_type target access

```
const type_info& target_type() const noexcept;
```

Returns: If `*this` has a target of type `T`, `typeid(T)`; otherwise, `typeid(void)`.

```
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
```

Returns: If `target_type() == typeid(Executor)` a pointer to the stored executor target; otherwise a null pointer value.

1.4.4.1.8 polymorphic_executor_type comparisons

```
template<class... SupportableProperties>
    bool operator==(const polymorphic_executor_type<SupportableProperties...> a, const polymorphic_executor_type<SupportableProperties...> b) noexcept;
```

Returns:

- true if !a and !b;
- true if a and b share a target;
- true if e and f are the same type and e == f, where e is the target of a and f is the target of b;
- otherwise false.

```
template<class... SupportableProperties>
bool operator==(const polymorphic_executor_type<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator==(nullptr_t, const polymorphic_executor_type<SupportableProperties...>& e) noexcept;
```

Returns: !e.

```
template<class... SupportableProperties>
bool operator!=(const polymorphic_executor_type<SupportableProperties...>& a, const polymorphic_executor_type<SupportableProperties...>& b) noexcept;
```

Returns: !(a == b).

```
template<class... SupportableProperties>
bool operator!=(const polymorphic_executor_type<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator!=(nullptr_t, const polymorphic_executor_type<SupportableProperties...>& e) noexcept;
```

Returns: (bool) e.

1.4.4.1.9 polymorphic_executor_type specialized algorithms

```
template<class... SupportableProperties>
void swap(polymorphic_executor_type<SupportableProperties...>& a, polymorphic_executor_type<SupportableProperties...>& b) noexcept;
```

Effects: a.swap(b).

```
template <class Property, class... SupportableProperties>
polymorphic_executor_type prefer(const polymorphic_executor_type<SupportableProperties...>& e, Property p);
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_preferable` is well-formed and has the value `true`.

Returns: A polymorphic wrapper whose target is the result of `execution::prefer(e, p)`, where e is the target object of *this.

1.4.4.1.10 polymorphic_executor_type casts

```
template<class... OtherSupportableProperties>
polymorphic_executor_type<OtherSupportableProperties...> static_executor_cast() const;
```

Requires: The target object was first inserted into a polymorphic wrapper (whether via the wrapper's constructor or assignment operator) whose template parameters included the parameters in `OtherSupportableProperties`.

Returns: A polymorphic wrapper whose target is e.

1.4.4.2 oneway_t customization points

In addition to conforming to the above specification for interface-changing properties, the `oneway_t` property provides the following customization:

```
struct oneway_t
{
    template<class Executor>
        friend see-below require(Executor ex, oneway_t);
};
```

This customization point returns an executor that satisfies the `oneway_t` requirements by adapting the native functionality of an executor that does not satisfy the `oneway_t` requirements.

```
template<class Executor>
friend see-below require(Executor ex, oneway_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` has member functions `require` and `query` that forward to the corresponding members of the copy of `ex`, if present. `e1` has the same properties as `ex`, except for the addition of the `oneway_t` property and the exclusion of other interface-changing properties. The type `E1` satisfies the `OneWayExecutor` requirements by implementing member function `execute` in terms of the member function `bulk_execute` of the object `ex`, and `E1` has a member function `Executor require(bulk_oneway_t) const` that returns a copy of `ex`.

Remarks: This function shall not participate in overload resolution unless `oneway_t::static_query_v<Executor>` is false and `bulk_oneway_t::static_query_v<Executor>` is true.

1.4.4.3 oneway_t polymorphic wrapper

In addition to conforming to the above specification for polymorphic wrappers, the nested class template `oneway_t::polymorphic_executor_type` provides the following member functions:

```
template <class... SupportableProperties>
class polymorphic_executor_type
{
public:
    template<class Executor>
        polymorphic_executor_type(Executor e);
```

```
template<class Executor>
    polymorphic_executor_type& operator=(Executor e);

template<class Function>
    void execute(Function&& f) const;
};
```

oneway_t::polymorphic_executor_type satisfies the OneWayExecutor requirements.

```
template<class Executor>
    polymorphic_executor_type(Executor e);
```

Remarks: This function shall not participate in overload resolution unless:

- can_require_v<Executor, oneway_t>.
- can_require_v<Executor, P>, if P::is_requirable, where P is each property in SupportableProperties...
- can_prefer_v<Executor, P>, if P::is_preferable, where P is each property in SupportableProperties...
- and can_query_v<Executor, P>, if P::is_requirable == false and P::is_preferable == false, where P is each property in SupportableProperties...

Effects: *this targets a copy of e1, where e1 is the result of execution::require(e, oneway).

```
template<class Executor>
    polymorphic_executor_type& operator=(Executor e);
```

Requires: As for template<class Executor> polymorphic_executor_type(Executor e).

Effects: polymorphic_executor_type(std::move(e)).swap(*this).

Returns: *this.

```
template<class Function>
    void execute(Function&& f) const;
```

Effects: Performs e.execute(f2), where:

- e is the target object of *this;
- f1 is the result of DECAY_COPY(std::forward<Function>(f));
- f2 is a function object of unspecified type that, when invoked as f2(), performs f1().

1.4.4.4 bulk_oneway_t customization points

In addition to conforming to the above specification for interface-changing properties, the oneway_t property provides the following customization:

```
struct bulk_oneway_t
{
    template<class Executor>
        friend see-below require(Executor ex, bulk_oneway_t);
};
```

This customization point returns an executor that satisfies the bulk_oneway_t requirements by adapting the native functionality of an executor that does not satisfy the bulk_oneway_t requirements.

```
template<class Executor>
    friend see-below require(Executor ex, bulk_oneway_t);
```

Returns: A value e1 of type E1 that holds a copy of ex. E1 has member functions require and query that forward to the corresponding members of the copy of ex, if present. e1 has the same properties as ex, except for the addition of the bulk_oneway_t property and the exclusion of other interface-changing properties. The type E1 satisfies the BulkOneWayExecutor requirements by implementing member function bulk_execute in terms of the member function execute of the object ex, and E1 has a member function Executor require(oneway_t) const that returns a copy of ex.

Remarks: This function shall not participate in overload resolution unless bulk_oneway_t::static_query_v<Executor> is false and oneway_t::static_query_v<Executor> is true.

1.4.4.5 bulk_oneway_t polymorphic wrapper

In addition to conforming to the above specification for polymorphic wrappers, the nested class template bulk_oneway_t::polymorphic_executor_type has the following member functions:

```
template <class... SupportableProperties>
class polymorphic_executor_type
{
public:
    template<class Executor>
        polymorphic_executor_type(Executor e);

    template<class Executor>
        polymorphic_executor_type& operator=(Executor e);

    template<class Function, class SharedFactory>
        void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
};
```

bulk_oneway_t::polymorphic_executor_type satisfies the BulkOneWayExecutor requirements.

```
template<class Executor>
    polymorphic_executor_type(Executor e);
```

Remarks: This function shall not participate in overload resolution unless:

- `can_require_v<Executor, bulk_oneway_t>`.
- `can_require_v<Executor, P>`, if `P::is_requirable`, where `P` is each property in `SupportableProperties...`
- `can_prefer_v<Executor, P>`, if `P::is_preferable`, where `P` is each property in `SupportableProperties...`
- and `can_query_v<Executor, P>`, if `P::is_requirable == false` and `P::is_preferable == false`, where `P` is each property in `SupportableProperties...`

Effects: *this targets a copy of `e1`, where `e1` is the result of `execution::require(e, bulk_oneway)`.

```
template<class Executor>
    polymorphic_executor_type& operator=(Executor e);
```

Requires: As for `template<class Executor> polymorphic_executor_type(Executor e)`.

Effects: `polymorphic_executor_type(std::move(e)).swap(*this)`.

Returns: *this.

```
template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Effects: Performs `e.bulk_execute(f2, n, sf2)`, where:

- `e` is the target object of *this;
- `sf1` is the result of `DECAY_COPY(std::forward<SharedFactory>(sf))`;
- `sf2` is a function object of unspecified type that, when invoked as `sf2()`, performs `sf1()`;
- `s1` is the result of `sf1()`;
- `s2` is the result of `sf2()`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when invoked as `f2(i, s2)`, performs `f1(i, s1)`, where `i` is a value of type `size_t`.

1.4.5 Behavioral properties

Behavioral properties define a set of mutually-exclusive nested properties describing executor behavior.

Unless otherwise specified, behavioral property types `S`, their nested property types `S::Ni`, and nested property objects `S::ni` conform to the following specification:

```
struct S
{
    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;
    using polymorphic_query_result_type = S;

    template<class Executor>
        static constexpr auto static_query_v
            = see-below;

    template<class Executor>
        friend constexpr S query(const Executor& ex, const Property& p) noexcept(see-below);

    friend constexpr bool operator==(const S& a, const S& b);
    friend constexpr bool operator!=(const S& a, const S& b) { return !operator==(a, b); }

    constexpr S();

    struct N1
    {
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = true;
        using polymorphic_query_result_type = S;

        template<class Executor>
            static constexpr auto static_query_v
                = see-below;

        static constexpr S value() { return S(N1()); }
    };

    static constexpr n1;

    constexpr S(const N1);

    ...

    struct NN
    {
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = true;
        using polymorphic_query_result_type = S;

        template<class Executor>
            static constexpr auto static_query_v
                = see-below;

        static constexpr S value() { return S(NN()); }
    };
};
```

```
static constexpr nN;

constexpr S(const NN);
};
```

Queries for the value of an executor's behavioral property shall not change between invocations unless the executor is assigned another executor with a different value of that behavioral property.

$S()$ and $S(S::Ei())$ are all distinct values of S . [*Note:* This means they compare unequal. *--end note.*]

The value returned from `execution::query(e1, p1)` and a subsequent invocation `execution::query(e1, p1)`, where

- $p1$ is an instance of S or $S::Ei$, and
- $e2$ is the result of `execution::require(e1, p2)` or `execution::prefer(e1, p2)`,

shall compare equal unless

- $p2$ is an instance of $S::Ei$, and
- $p1$ and $p2$ are different types.

The value of the expression $S::N1::static_query_v<Executor>$ is

- `Executor::query(S::N1())`, if that expression is a well-formed expression;
- ill-formed if `declval<Executor>().query(S::N1())` is well-formed;
- ill-formed if `can_query_v<Executor, S::Ni>` is true for any $1 < i \leq N$;
- otherwise $S::N1()$.

[*Note:* These rules automatically enable the $S::N1$ property by default for executors which do not provide a query function for properties $S::Ni$. *--end note*]

The value of the expression $S::Ni::static_query_v<Executor>$, for all $1 < i \leq N$, is

- `Executor::query(S::Ni())`, if that expression is a well-formed constant expression;
- otherwise ill-formed.

The value of the expression $S::static_query_v<Executor>$ is

- `Executor::query(S())`, if that expression is a well-formed constant expression;
- otherwise, ill-formed if `declval<Executor>().query(S())` is well-formed;
- otherwise, $S::Ni::static_query_v<Executor>$ for the least $i \leq N$ for which this expression is a well-formed constant expression;
- otherwise ill-formed.

[*Note:* These rules automatically enable the $S::N1$ property by default for executors which do not provide a query function for properties S or $S::Ni$. *--end note*]

Let k be the least value of i for which `can_query_v<Executor, S::Ni>` is true, if such a value of i exists.

```
template<class Executor>
    friend constexpr S query(const Executor& ex, const Property& p) noexcept(noexcept(execution::query(ex, std::declval<const S::Nk>())));
```

Returns: `execution::query(ex, S::Nk())`.

Remarks: This function shall not participate in overload resolution unless `is_same_v<Property, S> && can_query_v<Executor, S::Ni>` is true for at least one $S::Ni$.

```
bool operator==(const S& a, const S& b);
```

Returns: true if a and b were constructed from the same constructor; false, otherwise.

1.4.5.1 Blocking properties

The `blocking_t` property describes what guarantees executors provide about the blocking behavior of their execution functions.

`blocking_t` provides nested property types and objects as described below.

Nested Property Type	Nested Property Object Name	Requirements
<code>blocking_t::possibly_t</code>	<code>blocking_t::possibly</code>	Invocation of an executor's execution function may block pending completion of one or more invocations of the submitted function object.
<code>blocking_t::always_t</code>	<code>blocking_t::always</code>	Invocation of an executor's execution function shall block until completion of all invocations of submitted function object.
<code>blocking_t::never_t</code>	<code>blocking_t::never</code>	Invocation of an executor's execution function shall not block pending completion of the invocations of the submitted function object.

1.4.5.1.1 blocking_t::always_t customization points

In addition to conforming to the above specification, the blocking_t::always_t property provides the following customization:

```
struct always_t
{
    template<class Executor>
        friend see-below require(Executor ex, blocking_t::always_t);
};
```

If the executor has the blocking_adaptation_t::allowed_t property, this customization uses an adapter to implement the blocking_t::always_t property.

```
template<class Executor>
    friend see-below require(Executor ex, blocking_t::always_t);
```

Returns: A value e1 of type E1 that holds a copy of ex. If Executor satisfies the OneWayExecutor requirements, E1 shall satisfy the OneWayExecutor requirements by providing member functions require, query, and execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the BulkOneWayExecutor requirements, E1 shall satisfy the BulkOneWayExecutor requirements by providing member functions require, query, and bulk_execute that forward to the corresponding member functions of the copy of ex. In addition, E1 provides an overload of require such that e1.require(blocking.always) returns a copy of e1, an overload of query such that e1.query(blocking) returns blocking.always, and functions execute and bulk_execute shall block the calling thread until the submitted functions have finished execution. e1 has the same executor properties as ex, except for the addition of the blocking_t::always_t property, and removal of blocking_t::never_t and blocking_t::possibly_t properties if present.

Remarks: This function shall not participate in overload resolution unless blocking_adaptation_t::static_query_v<Executor> is blocking_adaptation.allowed.

1.4.5.2 Properties to indicate if blocking and directionality may be adapted

The blocking_adaptation_t property allows or disallows blocking or directionality adaptation via execution::require.

blocking_adaptation_t provides nested property types and objects as described below.

Nested Property Type	Nested Property Object Name	Requirements
blocking_adaptation_t::disallowed_t	blocking_adaptation::disallowed	The require customization point may not adapt the executor to add the blocking_t::always_t property.
blocking_adaptation_t::allowed_t	blocking_adaptation::allowed	The require customization point may adapt the executor to add the blocking_t::always_t property.

1.4.5.2.1 blocking_adaptation_t::allowed_t customization points

In addition to conforming to the above specification, the blocking_adaptation_t::allowed_t property provides the following customization:

```
struct allowed_t
{
    template<class Executor>
        friend see-below require(Executor ex, blocking_adaptation_t::allowed_t);
};
```

This customization uses an adapter to implement the blocking_adaptation_t::allowed_t property.

```
template<class Executor>
    friend see-below require(Executor ex, blocking_adaptation_t::allowed_t);
```

Returns: A value e1 of type E1 that holds a copy of ex. If Executor satisfies the OneWayExecutor requirements, E1 shall satisfy the OneWayExecutor requirements by providing member functions require, query, and execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the BulkOneWayExecutor requirements, E1 shall satisfy the BulkOneWayExecutor requirements by providing member functions require, query, and bulk_execute that forward to the corresponding member functions of the copy of ex. In addition, blocking_adaptation_t::static_query_v<E1> is blocking_adaptation.allowed, and e1.require(blocking_adaptation.disallowed) yields a copy of ex. e1 has the same executor properties as ex, except for the addition of the blocking_adaptation_t::allowed_t property.

1.4.5.3 Properties to indicate if submitted tasks represent continuations

The relationship_t property allows users of executors to indicate that submitted tasks represent continuations.

relationship_t provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
relationship_t::fork_t	relationship_t::fork	Function objects submitted through the executor do not represent continuations of the caller. Function objects submitted

```
relationship_t::continuation_t
```

```
relationship_t::continuation
```

through the executor represent continuations of the caller. Invocation of the submitted function object may be deferred until the caller completes.

1.4.5.4 Properties to indicate likely task submission in the future

The `outstanding_work_t` property allows users of executors to indicate that task submission is likely in the future.

`outstanding_work_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
outstanding_work_t::untracked_t	outstanding_work::untracked	The existence of the executor object does not indicate any likely future submission of a function object. The existence of the executor object represents an indication of likely future submission of a function object. The executor or its associated execution context may choose to maintain execution resources in anticipation of this submission.
outstanding_work_t::tracked_t	outstanding_work::tracked	

[**Note:** The `outstanding_work_t::tracked_t` and `outstanding_work_t::untracked_t` properties are used to communicate to the associated execution context intended future work submission on the executor. The intended effect of the properties is the behavior of execution context's facilities for awaiting outstanding work; specifically whether it considers the existence of the executor object with the `outstanding_work_t::tracked_t` property enabled outstanding work when deciding what to wait on. However this will be largely defined by the execution context implementation. It is intended that the execution context will define its wait facilities and on-destruction behaviour and provide an interface for querying this. An initial work towards this is included in P0737r0. *--end note*]

1.4.5.5 Properties for bulk execution guarantees

Bulk execution guarantee properties communicate the forward progress and ordering guarantees of execution agents associated with the bulk execution.

`bulk_guarantee_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
bulk_guarantee_t::unsequenced_t	bulk_guarantee_t::unsequenced	Execution agents within the same bulk execution may be parallelized and vectorized.
bulk_guarantee_t::sequenced_t	bulk_guarantee_t::sequenced	Execution agents within the same bulk execution may not be parallelized.
bulk_guarantee_t::parallel_t	bulk_guarantee_t::parallel	Execution agents within the same bulk execution may be parallelized.

Execution agents associated with the `bulk_guarantee_t::unsequenced_t` property may invoke the function object in an unordered fashion. Any such invocations in the same thread of execution are unsequenced with respect to each other. [Note: This means that multiple execution agents may be interleaved on a single thread of execution, which overrides the usual guarantee from [intro.execution] that function executions do not interleave with one another. *—end note*]

Execution agents associated with the `bulk_guarantee_t::sequenced_t` property invoke the function object in sequence in lexicographic order of their indices.

Execution agents associated with the `bulk_guarantee_t::parallel_t` property invoke the function object with a parallel forward progress guarantee. Any such invocations in the same thread of execution are indeterminately sequenced with respect to each other. [Note: It is the caller's responsibility to ensure that the invocation does not introduce data races or deadlocks. *--end note*]

[Editorial note: The descriptions of these properties were ported from [algorithms.parallel.user]. The intention is that a future standard will specify execution policy behavior in terms of the fundamental properties of their associated executors. We did not include the accompanying code examples from [algorithms.parallel.user] because the examples seem easier to understand when illustrated by `std::for_each`. --end editorial note]

1.4.5.6 Properties for mapping of execution on to threads

The `mapping_t` property describes what guarantees executors provide about the mapping of execution agents onto threads of execution.

mapping_t provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
mapping_t::thread_t	mapping::thread	Execution agents are mapped onto threads of execution.
mapping_t::new_thread_t	mapping::new_thread	Each execution agent is mapped onto a new thread of execution.
mapping_t::other_t	mapping::other	Mapping of each execution agent is implementation-defined.

[*Note*: A mapping of an execution agent onto a thread of execution implies the execution agent runs as-if on a `std::thread`. Therefore, the facilities provided by `std::thread`, such as thread-local storage, are available. `mapping_t::new_thread_t` provides stronger guarantees, in particular that thread-local storage will not be shared between execution agents. --*end note*]

1.4.6 Properties for customizing memory allocation

```
template <typename ProtoAllocator>
struct allocator_t;
```

The `allocator_t` property conforms to the following specification:

```
template <typename ProtoAllocator>
struct allocator_t
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;

    template<class Executor>
    static constexpr auto static_query_v
        = Executor::query(allocator_t);

    template <typename OtherProtoAllocator>
    allocator_t<OtherProtoAllocator> operator()(const OtherProtoAllocator &a) const;

    static constexpr ProtoAllocator value() const;

private:
    ProtoAllocator a_; // exposition only
};
```

Property	Notes	Requirements
allocator_t<ProtoAllocator>	Result of allocator_t<void>::operator(OtherProtoAllocator).	For executor types that satisfy the <code>OneWayExecutor</code> requirements, the executor implementation shall use the encapsulated allocator to allocate any memory required to store the submitted function object. For executor types that satisfy the <code>OneWayExecutor</code> requirements, the executor implementation shall use an implementation defined default allocator to allocate any memory required to store the submitted function object.
allocator_t<void>	Specialisation of <code>allocator_t<ProtoAllocator></code> .	

If the expression `execution::query(E, P)` is well formed, where `P` is an object of type `allocator_t<ProtoAllocator>`, then: * the type of the expression

execution::query(E, P) shall satisfy the ProtoAllocator requirements; * the result of the expression execution::query(E, P) shall be the allocator currently established in the executor E; and * the expression execution::query(E, allocator_t<void>{}) shall also be well formed and have the same result as execution::query(E, P).

1.4.6.1 allocator_t members

```
template <typename OtherProtoAllocator>
allocator_t<OtherProtoAllocator> operator()(const OtherProtoAllocator &a) const;
```

Returns: An allocator object whose exposition-only member a_ is initialized as a_(a).

Remarks: This function shall not participate in overload resolution unless ProtoAllocator is void.

[Note: It is permitted for a to be an executor’s implementation-defined default allocator and, if so, the default allocator may also be established within an executor by passing the result of this function to require. --end note]

```
static constexpr ProtoAllocator value() const;
```

Returns: The exposition-only member a_.

Remarks: This function shall not participate in overload resolution unless ProtoAllocator is not void.

1.5 Executor type traits

1.5.1 Determining that a type satisfies executor type requirements

```
template<class T> struct is_oneway_executor;
template<class T> struct is_bulk_oneway_executor;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a UnaryTypeTrait (C++Std [meta.rqmts]) with a BaseCharacteristic of true_type if the corresponding condition is true, otherwise false_type.

Template	Condition	Preconditions
template<class T> struct is_oneway_executor	⊢ meets the syntactic requirements for OneWayExecutor.	⊢ is a complete type.
template<class T> struct is_bulk_oneway_executor	⊢ meets the syntactic requirements for BulkOneWayExecutor.	⊢ is a complete type.

1.5.2 Associated shape type

```
template<class Executor>
struct executor_shape
{
    private:
        // exposition only
        template<class T>
        using helper = typename T::shape_type;

    public:
        using type = std::experimental::detected_or_t<
            size_t, helper, decltype(execution::require(declval<const Executor&>(), execution::bulk))
        >;

        // exposition only
        static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};
```

1.5.3 Associated index type

```
template<class Executor>
struct executor_index
{
    private:
        // exposition only
        template<class T>
        using helper = typename T::index_type;

    public:
        using type = std::experimental::detected_or_t<
            executor_shape_t<Executor>, helper, decltype(execution::require(declval<const Executor&>(), execution::bulk))
        >;

        // exposition only
        static_assert(std::is_integral_v<type>, "index type must be an integral type");
};
```

1.6 Polymorphic executor support

1.6.1 Class bad_executor

An exception of type bad_executor is thrown by polymorphic executor member functions execute and bulk_execute when the executor object has no target.

```
class bad_executor : public exception
{
public:
    // constructor:
    bad_executor() noexcept;
};
```

1.6.1.1 bad_executor constructors

```
bad_executor() noexcept;
```

Effects: Constructs a `bad_executor` object.

Postconditions: `what()` returns an implementation-defined NTBS.

1.6.2 Struct `prefer_only`

The `prefer_only` struct is a property adapter that disables the `is_requirable` value.

[Example:

Consider a generic function that performs some task immediately if it can, and otherwise asynchronously in the background.

```
template<class Executor>
void do_async_work(
    Executor ex,
    Callback callback)
{
    if (try_work() == done)
    {
        // Work completed immediately, invoke callback.
        execution::require(ex,
            execution::single,
            execution::oneway,
            ).execute(callback);
    }
    else
    {
        // Perform work in background. Track outstanding work.
        start_background_work(
            execution::prefer(ex,
                execution::outstanding_work.tracked),
            callback);
    }
}
```

This function can be used with an inline executor which is defined as follows:

```
struct inline_executor
{
    constexpr bool operator==(const inline_executor&) const noexcept
    {
        return true;
    }

    constexpr bool operator!=(const inline_executor&) const noexcept
    {
        return false;
    }

    template<class Function> void execute(Function f) const noexcept
    {
        f();
    }
};
```

as, in the case of an unsupported property, invocation of `execution::prefer` will fall back to an identity operation.

The polymorphic executor wrapper should be able to simply swap in, so that we could change `do_async_work` to the non-template function:

```
void do_async_work(
    executor<
        execution::single,
        execution::oneway,
        execution::outstanding_work_t::tracked_t> ex,
    std::function<void()> callback)
{
    if (try_work() == done)
    {
        // Work completed immediately, invoke callback.
        execution::require(ex,
            execution::single,
            execution::oneway,
            ).execute(callback);
    }
    else
    {
        // Perform work in background. Track outstanding work.
        start_background_work(
            execution::prefer(ex,
```

```

        execution::outstanding_work.tracked),
        callback);
    }
}

```

with no change in behavior or semantics.

However, if we simply specify `execution::outstanding_work.tracked` in the executor template parameter list, we will get a compile error due to the executor template not knowing that `execution::outstanding_work.tracked` is intended for use with `prefer` only. At the point of construction from an `inline_executor` called `ex`, `executor` will try to instantiate implementation templates that perform the ill-formed `execution::require(ex, execution::outstanding_work.tracked)`.

The `prefer_only` adapter addresses this by turning off the `is_requirable` attribute for a specific property. It would be used in the above example as follows:

```

void do_async_work(
    executor<
        execution::single,
        execution::oneway,
        prefer_only<execution::outstanding_work_t::tracked_t>> ex,
        std::function<void()> callback)
{
    ...
}

```

-- end example]

```

template<class InnerProperty>
struct prefer_only
{
    InnerProperty property;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = InnerProperty::is_preferable;

    using polymorphic_query_result_type = see-below; // not always defined

    template<class Executor>
        static constexpr auto static_query_v = see-below; // not always defined

    constexpr prefer_only(const InnerProperty& p);

    constexpr auto value() const
        noexcept(noexcept(std::declval<const InnerProperty>().value()))
        -> decltype(std::declval<const InnerProperty>().value());

    template<class Executor, class Property>
    friend auto prefer(Executor ex, const Property& p)
        noexcept(noexcept(execution::prefer(std::move(ex), std::declval<const InnerProperty>())))
        -> decltype(execution::prefer(std::move(ex), std::declval<const InnerProperty>()));

    template<class Executor, class Property>
    friend constexpr auto query(const Executor& ex, const Property& p)
        noexcept(noexcept(execution::query(ex, std::declval<const InnerProperty>())))
        -> decltype(execution::query(ex, std::declval<const InnerProperty>()));
};

```

If `InnerProperty::polymorphic_query_result_type` is valid and denotes a type, the template instantiation `prefer_only<InnerProperty>` defines a nested type `polymorphic_query_result_type` as a synonym for `InnerProperty::polymorphic_query_result_type`.

If `InnerProperty::static_query_v` is a variable template and `InnerProperty::static_query_v<E>` is well formed for some executor type `E`, the template instantiation `prefer_only<InnerProperty>` defines a nested variable template `static_query_v` as a synonym for `InnerProperty::static_query_v`.

```
constexpr prefer_only(const InnerProperty& p);
```

Effects: Initializes property with `p`.

```

constexpr auto value() const
    noexcept(noexcept(std::declval<const InnerProperty>().value()))
    -> decltype(std::declval<const InnerProperty>().value());

```

Returns: `property.value()`.

Remarks: Shall not participate in overload resolution unless the expression `property.value()` is well-formed.

```

template<class Executor, class Property>
friend auto prefer(Executor ex, const Property& p)
    noexcept(noexcept(execution::prefer(std::move(ex), std::declval<const InnerProperty>())))
    -> decltype(execution::prefer(std::move(ex), std::declval<const InnerProperty>()));

```

Returns: `execution::prefer(std::move(ex), p.property)`.

Remarks: Shall not participate in overload resolution unless `std::is_same_v<Property, prefer_only>` is true, and the expression `execution::prefer(std::move(ex), p.property)` is well-formed.

```

template<class Executor, class Property>
friend constexpr auto query(const Executor& ex, const Property& p)
    noexcept(noexcept(execution::query(ex, std::declval<const InnerProperty>())))
    -> decltype(execution::query(ex, std::declval<const InnerProperty>()));

```

Returns: `execution::query(ex, p.property)`.

Remarks: Shall not participate in overload resolution unless `std::is_same_v<Property, prefer_only>` is true, and the expression `execution::query(ex, p.property)` is well-formed.

1.7 Thread pools

Thread pools manage execution agents which run on threads without incurring the overhead of thread creation and destruction whenever such agents are needed.

1.7.1 Header <thread_pool> synopsis

```
namespace std {  
  
    class static_thread_pool;  
  
} // namespace std
```

1.7.2 Class static_thread_pool

`static_thread_pool` is a statically-sized thread pool which may be explicitly grown via thread attachment. The `static_thread_pool` is expected to be created with the use case clearly in mind with the number of threads known by the creator. As a result, no default constructor is considered correct for arbitrary use cases and `static_thread_pool` does not support any form of automatic resizing.

`static_thread_pool` presents an effectively unbounded input queue and the execution functions of `static_thread_pool`'s associated executors do not block on this input queue.

[*Note:* Because `static_thread_pool` represents work as parallel execution agents, situations which require concurrent execution properties are not guaranteed correctness. *--end note.*]

```
class static_thread_pool  
{  
    public:  
        using executor_type = see-below;  
  
        // construction/destruction  
        explicit static_thread_pool(std::size_t num_threads);  
  
        // nocopy  
        static_thread_pool(const static_thread_pool&) = delete;  
        static_thread_pool& operator=(const static_thread_pool&) = delete;  
  
        // stop accepting incoming work and wait for work to drain  
        ~static_thread_pool();  
  
        // attach current thread to the thread pools list of worker threads  
        void attach();  
  
        // signal all work to complete  
        void stop();  
  
        // wait for all threads in the thread pool to complete  
        void wait();  
  
        // placeholder for a general approach to getting executors from  
        // standard contexts.  
        executor_type executor() noexcept;  
};
```

For an object of type `static_thread_pool`, *outstanding work* is defined as the sum of:

- the number of existing executor objects associated with the `static_thread_pool` for which the `execution::outstanding_work.tracked` property is established;
- the number of function objects that have been added to the `static_thread_pool` via the `static_thread_pool` executor, but not yet invoked; and
- the number of function objects that are currently being invoked within the `static_thread_pool`.

The `static_thread_pool` member functions `executor`, `attach`, `wait`, and `stop`, and the associated executors' copy constructors and member functions, do not introduce data races as a result of concurrent invocations of those functions from different threads of execution.

A `static_thread_pool`'s threads run execution agents with forward progress guarantee delegation. [*Note:* Forward progress is delegated to an execution agent for its lifetime. Because `static_thread_pool` guarantees only parallel forward progress to running execution agents; *i.e.*, execution agents which have run the first step of the function object. *--end note*]

1.7.2.1 Types

using executor_type = see-below;

An executor type conforming to the specification for `static_thread_pool` executor types described below.

1.7.2.2 Construction and destruction

```
static_thread_pool(std::size_t num_threads);
```

Effects: Constructs a `static_thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`.

```
~static_thread_pool();
```

Effects: Destroys an object of class `static_thread_pool`. Performs `stop()` followed by `wait()`.

1.7.2.3 Worker management

```
void attach();
```

Effects: Adds the calling thread to the pool such that this thread is used to execute submitted function objects. [*Note:* Threads created during thread pool construction, or previously attached to the pool, will continue to be used for function object execution. *--end note*] Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `static_thread_pool` object construction have completed. (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. Invocation of `stop()` returns without waiting for the threads to complete. Subsequent invocations to `attach` complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is $\emptyset$. Blocks the calling thread (C++Std [defns.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent invocations of `attach()` complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

1.7.2.4 Executor creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool. The returned executor has the following properties already established:

- `execution::oneway`
- `execution::blocking.possibly`
- `execution::relationship.fork`
- `execution::outstanding_work.untracked`
- `execution::allocator`
- `execution::allocator(std::allocator<void>())`

1.7.3 static_thread_pool executor types

All executor types accessible through `static_thread_pool::executor()`, and subsequent invocations of the member function `require`, conform to the following specification.

```
class C
{
public:
    // types:

    typedef std::size_t shape_type;
    typedef std::size_t index_type;

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // executor operations:

    see-below require(execution::blocking_t::never_t) const;
    see-below require(execution::blocking_t::possibly_t) const;
    see-below require(execution::blocking_t::always_t) const;
    see-below require(execution::relationship_t::continuation_t) const;
    see-below require(execution::relationship_t::fork_t) const;
    see-below require(execution::outstanding_work_t::tracked_t) const;
    see-below require(execution::outstanding_work_t::untracked_t) const;
    see-below require(const execution::allocator_t<void>& a) const;
    template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

    static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t::parallel_t) const;
    static constexpr execution::mapping_t query(execution::mapping_t::thread_t) const;
    execution::blocking_t query(execution::blocking_t) const;
    execution::relationship_t query(execution::relationship_t) const;
    execution::outstanding_work_t query(execution::outstanding_work_t) const;
    see-below query(execution::context_t) const noexcept;
    see-below query(execution::allocator_t<void>) const noexcept;
    template<class ProtoAllocator>
```

see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

```
    bool running_in_this_thread() const noexcept;
};
```

```
bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;
```

Objects of type `c` are associated with a `static_thread_pool`.

1.7.3.1 Constructors

```
C(const C& other) noexcept;
```

Postconditions: `*this == other`.

```
C(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

1.7.3.2 Assignment

```
C& operator=(const C& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
C& operator=(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

1.7.3.3 Operations

```
see-below require(execution::blocking_t::never_t) const;
see-below require(execution::blocking_t::possibly_t) const;
see-below require(execution::blocking_t::always_t) const;
see-below require(execution::relationship_t::continuation_t) const;
see-below require(execution::relationship_t::fork_t) const;
see-below require(execution::outstanding_work_t::tracked_t) const;
see-below require(execution::outstanding_work_t::untracked_t) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of `*this`.

```
see-below require(const execution::allocator_t<void>& a) const;
```

Returns: `require(execution::allocator(x))`, where `x` is an implementation-defined default allocator.

```
template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, with the `execution::allocator_t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.alloc`. All other properties of the returned executor object are identical to those of `*this`.

```
static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t) const;
```

Returns: `execution::bulk_guarantee.parallel`

```
static constexpr execution::mapping_t query(execution::mapping_t) const;
```

Returns: `execution::mapping.thread`.

```
execution::blocking_t query(execution::blocking_t) const;
execution::relationship_t query(execution::relationship_t) const;
execution::outstanding_work_t query(execution::outstanding_work_t) const;
```

Returns: The value of the given property of `*this`.

```
static_thread_pool& query(execution::context_t) const noexcept;
```

Returns: A reference to the associated `static_thread_pool` object.

```
see-below query(execution::allocator_t<void>) const noexcept;
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;
```

Returns: The allocator object associated with the executor, with type and value as either previously established by the `execution::allocator_t<ProtoAllocator>` property or the implementation defined default allocator established by the `execution::allocator_t<void>` property.

```
bool running_in_this_thread() const noexcept;
```

Returns: `true` if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

1.7.3.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: true if `&a.query(execution::context) == &b.query(execution::context)` and `a` and `b` have identical properties, otherwise false.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: `!(a == b)`.

1.7.3.5 static_thread_pool executor types with the execution::oneway property

In addition to conforming to the above specification, `static_thread_pool` executors having the `execution::oneway` property established shall conform to the following specification.

```
class C
{
public:
    template<class Function>
        void execute(Function&& f) const;
};
```

`C` is a type satisfying the `OneWayExecutor` requirements.

```
template<class Function>
    void execute(Function&& f) const;
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `OneWayExecutor` requirements and the properties established for `*this`. If the submitted function `f` exits via an exception, the `static_thread_pool` invokes `std::terminate()`.

1.7.3.6 static_thread_pool executor types with the execution::bulk_oneway property

In addition to conforming to the above specification, `static_thread_pool` executors having the `execution::bulk_oneway` property established shall conform to the following specification.

```
class C
{
public:
    template<class Function, class SharedFactory>
        void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
};
```

`C` is a type satisfying the `BulkOneWayExecutor` requirements.

```
template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkOneWayExecutor` requirements and the properties established for `*this`. If the submitted function `f` exits via an exception, the `static_thread_pool` invokes `std::terminate()`.